LessonForge — Complete Solution

> The whole project, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

The final layout

```
<repo folder>/
├── data/library/
│   ├── chunking.md
│   └── rag_basics.md
├── src/
│   ├── models/__init__.py, llm.py
│   ├── rag/__init__.py, library_store.py, search.py
│   ├── utils/__init__.py, logger.py
│   ├── __init__.py
│   ├── builder.py
│   ├── config.py
│   ├── exceptions.py
│   ├── lesson.py
│   ├── pipeline.py
│   ├── planner.py
│   └── schemas.py
├── tests/
│   ├── integration/test_live_lesson.py
│   └── unit/test_config.py, test_lesson.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

Setup files

`requirements.txt`:

```
chromadb
langchain-community
langchain-openai
langchain-text-splitters
openai
pydantic
python-dotenv
```

`.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`. `LICENSE` is the MIT text. Every folder under `src/` gets an empty `__init__.py`.

Sample content library (`data/library/`)

```
# data/library/rag_basics.md
Retrieval-augmented generation (RAG) first retrieves relevant documents for a
question, then generates an answer grounded in them. Embeddings turn text into
vectors so that similar meanings sit close together, which is how the search
finds relevant documents.
```

```
# data/library/chunking.md
Chunking splits a document into pieces before embedding. Where you split
matters: cut a fact in half and neither chunk can answer a question about it.
Recursive character splitting tries natural boundaries first.
```

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
```

```
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

from src.config import Config

def get_model(config: Config) -> ChatOpenAI:
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The typed shapes, the builder, the compiler

```
# src/schemas.py
from typing import List

from pydantic import BaseModel

class SubTopics(BaseModel):
    sub_topics: List[str] = []

class Section(BaseModel):
    sub_topic: str
    explanation: str
    source: str

class Lesson(BaseModel):
    topic: str
    sections: List[Section] = []
```

```
# src/builder.py
# Run the search() for each sub-topic and collect a Section. search() is passed
# in - a RAG lookup in the pipeline, a fake one in tests - so the engine needs
# no model or embeddings to test.
from src.schemas import Section

def build_sections(sub_topics, search) -> list:
    sections = []
    for sub_topic in sub_topics:
        text, source = search(sub_topic)
        sections.append(Section(
            sub_topic=sub_topic, explanation=text, source=source,
        ))
    return sections
```

```
# src/lesson.py
# Fold the sections into a Lesson and format it with numbered citations.
# Sources are numbered in first-seen order and de-duplicated.
from src.schemas import Lesson

def compile_lesson(topic: str, sections) -> Lesson:
    return Lesson(topic=topic, sections=sections)

def format_lesson(lesson: Lesson) -> str:
    sources = []
```

```
    for section in lesson.sections:
        if section.source not in sources:
            sources.append(section.source)

    lines = [f"Lesson: {lesson.topic}", ""]
    number = 1
    for section in lesson.sections:
        cite = sources.index(section.source) + 1
        lines.append(f"{number}. {section.sub_topic}")
        lines.append(f"   {section.explanation} [{cite}]")
        number += 1
    lines.append("")
    lines.append("Sources:")
    index = 1
    for source in sources:
        lines.append(f"  [{index}] {source}")
        index += 1
    return "\n".join(lines)
```

The planner and the RAG search

```
# src/planner.py
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import SubTopics

PLAN_PROMPT = (
    "Break the topic below into 3-5 sub-topics that make a clear lesson. "
    "Return just the sub-topics as short titles.\n\nTopic: {topic}"
)

def make_plan(model, topic: str) -> SubTopics:
    prompt = ChatPromptTemplate.from_template(PLAN_PROMPT)
    chain = prompt | model.with_structured_output(SubTopics)
    return chain.invoke({"topic": topic})
```

```
# src/rag/library_store.py
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

import chromadb

COLLECTION_NAME = "library"
```

```
EMBEDDINGS = OpenAIEmbeddings(model="text-embedding-3-small")

def build_library_store(library_dir: str = "data/library"):
    loader = DirectoryLoader(library_dir, glob="**/*.md",
                             loader_cls=TextLoader)
    documents = loader.load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=400, chunk_overlap=40)
    chunks = splitter.split_documents(documents)

    client = chromadb.Client()
    try:
        client.delete_collection(COLLECTION_NAME)
    except Exception:
        pass
    collection = client.create_collection(COLLECTION_NAME)

    ids = []
    texts = []
    metadatas = []
    for index in range(len(chunks)):
        ids.append(f"chunk-{index}")
        texts.append(chunks[index].page_content)
        source = chunks[index].metadata.get("source", "unknown")
        metadatas.append({"source": source.split("/")[-1]})
    collection.add(ids=ids, documents=texts, metadatas=metadatas,
                   embeddings=EMBEDDINGS.embed_documents(texts))
    return collection
```

```
# src/rag/search.py
from src.rag.library_store import EMBEDDINGS

def make_search(collection):
    def search(query: str):
        if collection.count() == 0:
            return "No content available.", "none"
        results = collection.query(
            query_embeddings=[EMBEDDINGS.embed_query(query)], n_results=1,
        )
        documents = results["documents"][0]
        metadatas = results["metadatas"][0]
        if not documents:
            return "No information found.", "none"
        return documents[0], metadatas[0].get("source", "unknown")

    return search
```

The pipeline and main

```
# src/pipeline.py
from src.builder import build_sections
from src.lesson import compile_lesson
from src.planner import make_plan
from src.rag.library_store import build_library_store
from src.rag.search import make_search
from src.schemas import Lesson
from src.utils.logger import get_logger

def run_lesson(model, topic: str) -> Lesson:
    logger = get_logger(__name__)
    plan = make_plan(model, topic).sub_topics
    logger.info(f"planned {len(plan)} sub-topic(s)")

    collection = build_library_store()
    search = make_search(collection)

    sections = build_sections(plan, search)
    return compile_lesson(topic, sections)
```

```
# main.py
import sys

from src.config import load_config
from src.lesson import format_lesson
from src.models.llm import get_model
from src.pipeline import run_lesson

def main():
    if len(sys.argv) < 2:
        print('usage: python main.py "your topic"')
        return
    topic = sys.argv[1]
    config = load_config()
    model = get_model(config)
    lesson = run_lesson(model, topic)
    print()
    print(format_lesson(lesson))

if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "LESSONFORGE_TEST_SETTING"

def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_lesson.py
# Run from the repo root: python -m tests.unit.test_lesson
from src.builder import build_sections
from src.lesson import compile_lesson, format_lesson

TABLE = [
    ("rag", ("RAG retrieves then generates.", "rag_basics.md")),
    ("chunk", ("Chunking splits documents before embedding.", "chunking.md")),
    ("embedding", ("Embeddings turn text into vectors.", "rag_basics.md")),
]

def fake_search(query: str):
    lowered = query.lower()
```

```
    for key, value in TABLE:
        if key in lowered:
            return value
    return ("No information found.", "none")

def build_lesson():
    plan = ["What is RAG?", "How does chunking work?", "What is an embedding?"]
    sections = build_sections(plan, fake_search)
    return compile_lesson("RAG", sections)

def test_sections_appear() -> None:
    text = format_lesson(build_lesson())
    result = "OK"
    if "Chunking splits documents" not in text:
        result = "WRONG"
    print(f"case 1 (sections in lesson)      -> {result}")

def test_citations_dedup() -> None:
    text = format_lesson(build_lesson())
    result = "OK"
    if "[1] rag_basics.md" not in text or "[2] chunking.md" not in text:
        result = "WRONG"
    if text.count("[1]") < 2:          # RAG and embedding share rag_basics.md
        result = "WRONG"
    print(f"case 2 (citations numbered/dedup) -> {result}")

if __name__ == "__main__":
    test_sections_appear()
    test_citations_dedup()
```

```
# tests/integration/test_live_lesson.py
# Real model + RAG. Run from the repo root:
#   python -m tests.integration.test_live_lesson
from src.config import load_config
from src.lesson import format_lesson
from src.models.llm import get_model
from src.pipeline import run_lesson

def test_lesson_has_cited_sections() -> None:
    config = load_config()
    model = get_model(config)
    lesson = run_lesson(model, "retrieval-augmented generation")
    text = format_lesson(lesson)
    ok = len(lesson.sections) >= 2 and "[1]" in text and "Sources:" in text
    print(f"case 1 (several cited sections) -> {'OK' if ok else 'WRONG'}")
```

```
if __name__ == "__main__":
    test_lesson_has_cited_sections()
```

Expected unit output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (sections in lesson)      -> OK
case 2 (citations numbered/dedup) -> OK
```

The whole project, in one sentence

The topic planned into sub-topics, each researched against your content library, and the sections compiled into a lesson where every part is cited — the same plan-research-cite patterns as ResearchPilot, pointed at building lessons.